



机器学习实验报告

回归实验

题目: CS:GO 选手 rating 回归任务

姓名: 汪君哲 许思源

时间: 2025-10-04

CS:GO 选手 rating 回归任务

1 背景介绍

在机器学习研究领域中，传统回归算法（如最小二乘线性回归、岭回归、Lasso 回归和 Elastic Net 等）以参数可解释性强、样本效率高以及训练稳定为主要优势，被广泛应用于连续变量预测、因果线索探索以及高维数据处理。这些算法的核心思想在于基于明确的函数假设（如线性或多项式关系）和正则化约束，通过最小化经验风险（如均方误差损失函数）并抑制过拟合来构建鲁棒模型。相比于黑箱式的深度学习方法，传统回归模型的优势在于其参数（如系数）直接对应特征的重要性，便于解释和诊断模型行为，尤其在小样本数据集或需要因果推理的场景中表现出色。

CS:GO (Counter-Strike: Global Offensive) 比赛已走过多个年头，在无数天才少年横空出世的光芒下，那些曾经在赛场上叱咤风云的老选手逐渐淡出视线。比较选手实力是人类天性使然，更是玩家们津津乐道的谈资。然而，随着游戏的发展，选手评分规则 (Rating 的计算方式) 不断演变，导致玩家难以找到一个凌驾于时空之上的统一评判标准。

因此，本项目以此为应用背景，基于 Ridge 回归、Lasso 回归和 Elastic Net 构建回归模型，旨在拟合 Rating 2.0 的函数形式，从而在统一标准下计算不同时代选手的评分，实现新老选手的公平比较。通过 K 折交叉验证进行超参数调优，并在独立测试集上使用 MSE 和 MAE 评估性能，该方法不仅强化了对传统回归算法的理解，还为电子竞技领域的定量分析提供了一个可复用的框架，突显正则化在平衡偏差-方差权衡中的作用，适用于学术研究和实际应用。

2 实验过程

2.1 数据准备

为了完成这个任务，我们从 Kaggle (www.kaggle.com) 上获取数据集。选取了浏览量和数据量较大的数据集 “CSGO: Professional Players Dataset”

(<https://www.kaggle.com/datasets/naumanaarif/csgo-pro-players-dataset/data>)。它涵盖了全球范围内 803 名专业玩家的历史统计数据。数据来源主要是从 www.hltv.org (一个知名的 CS:GO 赛事和玩家统计网站) 爬取而来，时间跨度为玩家的全部职业生涯，地理范围覆盖全球。

在拿到数据后，我们先读取了数据的基本信息，该表格形状为(803, 20)，数值型数据列一共有 16 列，分别是

```
['maps_played', 'rounds_played', 'kd_difference', 'kd_ratio', 'rating', 'total_kills',
```

'headshot_percentage', 'total_deaths', 'grenade_damage_per_round', 'kills_per_round',
'assists_per_round', 'deaths_per_round', 'teammate_saved_per_round',
'saved_by_teammate_per_round', 'kast', 'impact']

非数值型数据列一共有 4 列，分别是

['nick', 'country', 'stats_link', 'teams']

2.2 数据清洗

我们数据清洗的第一步是 null 值检查，发现整个表格中不存在空值。随后我们进行了选手昵称唯一性检查，发现唯一昵称数量为 801，重复的昵称数量为 2，于是我们先删除了重复行。

在简单清洗之后，我们进行了数据分布可视化和相关性分析

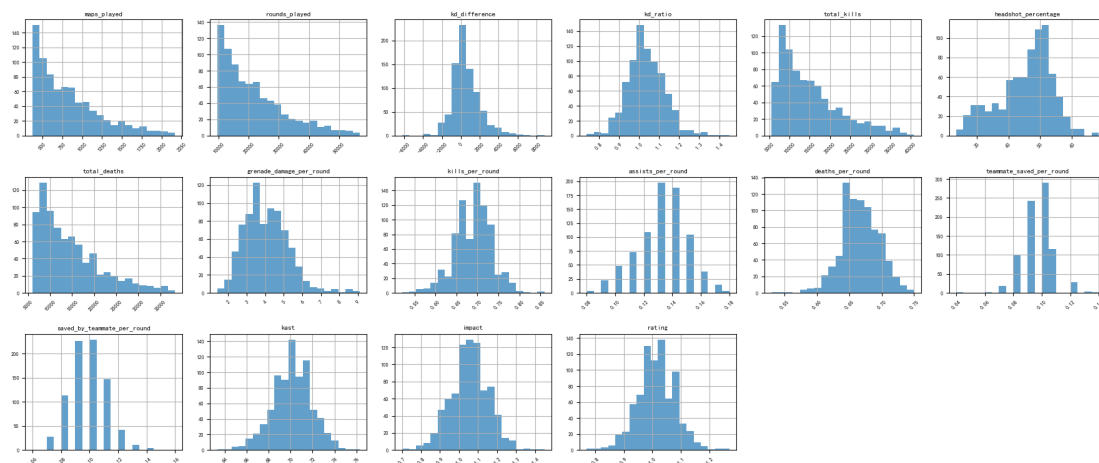


图 1 数据分布可视化

可以看出['maps_played', 'rounds_played', 'total_kills', 'total_deaths']近似长尾分布，这种分布常见于电子竞技数据，反映了选手参与度和表现的异质性（例如，新手选手比赛场次少，而职业选手积累大量数据）。其余特征，如 K/D ratio、ADR 和 Rating 2.0 等，近似正态分布，呈现出对称的钟形曲线。这表明这些指标在选手群体中较为均匀分布，便于线性假设的回归模型应用。

接下来，我们计算了特征间的 Pearson 相关系数矩阵，并通过热力图（Heatmap）进行可视化（图 2），以评估变量间的线性相关性。Pearson 相关系数 $\rho_{X,Y}$ 的计算公式为：

$$\rho_{X,Y} = \frac{Cov(X,Y)}{\sigma_X \sigma_Y} = \frac{E[(X - \mu_X)(Y - \mu_Y)]}{\sqrt{E[(X - \mu_X)^2]} \cdot \sqrt{E[(Y - \mu_Y)^2]}}$$

其中， $Cov(X,Y)$ 表示变量 X 和 Y 的协方差， σ_X 和 σ_Y 分别为 X 和 Y 的标准差， μ_X 和 μ_Y 为均值。该系数范围为 $[-1,1]$ ，值越接近 1 表示正相关越强，接近 -1 表示负相关越强，接近 0 表示无关。

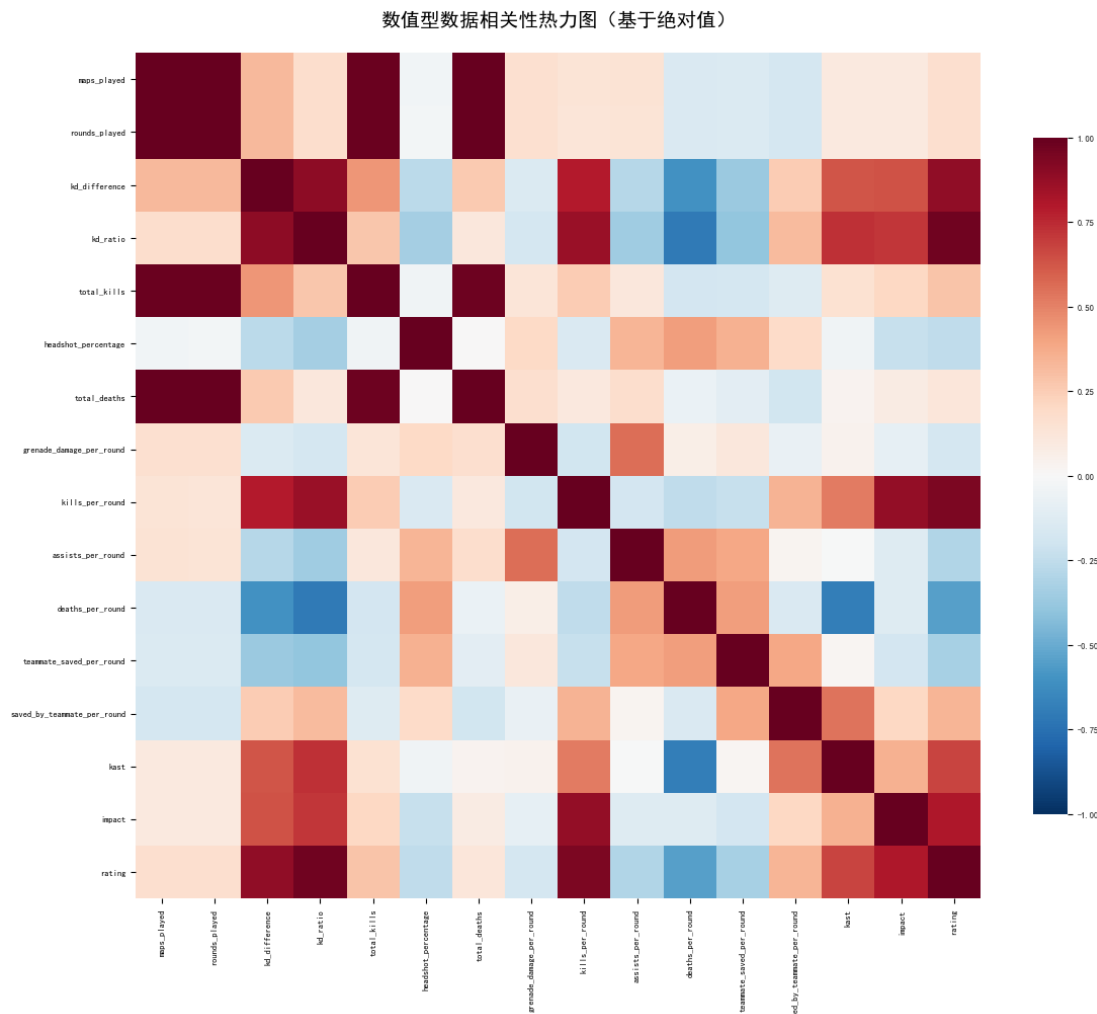


图 2 特征间的 Pearson 相关系数矩阵

分析结果显示：Rating 2.0 与 kills_per_round（每回合击杀数）和 K/D ratio（击杀死亡比）相关系数也在 0.7 之间，表明这些是 Rating 计算的核心影响因素，支持我们使用回归模型拟合 Rating 2.0 函数的假设。Rating 与击杀死亡差值（K/D Diff）、每回合死亡数（DPR）、KAST（击杀/助攻/存活/补枪综合指标）以及 Impact 也存在较强的关联性。

total_kills 与 rounds_played 呈现高度正相关（相关系数 $\rho > 0.8$ ），这符合直观预期——更多回合通常伴随更多击杀机会。这种多重共线性可能导致 OLS 回归系数不稳定，因此 Ridge 回归的 L2 正则化将成为首选，以收缩系数并维持模型鲁棒性。

某些辅助特征（如选手年龄或队伍排名）与核心指标相关性较低（ $|\rho| < 0.3$ ），Lasso 回归可自动将其系数置零，实现特征选择。这通过公式体现为低协方差，导致 ρ 接近 0。

相关性分析不仅揭示了数据内在结构，还为 Elastic Net 的混合正则化提供了依据：在存在相关特征组时，Elastic Net 能平衡稀疏性和稳定性。

为了更直观显示数据间的相关性，我们进一步绘制了散点矩阵。

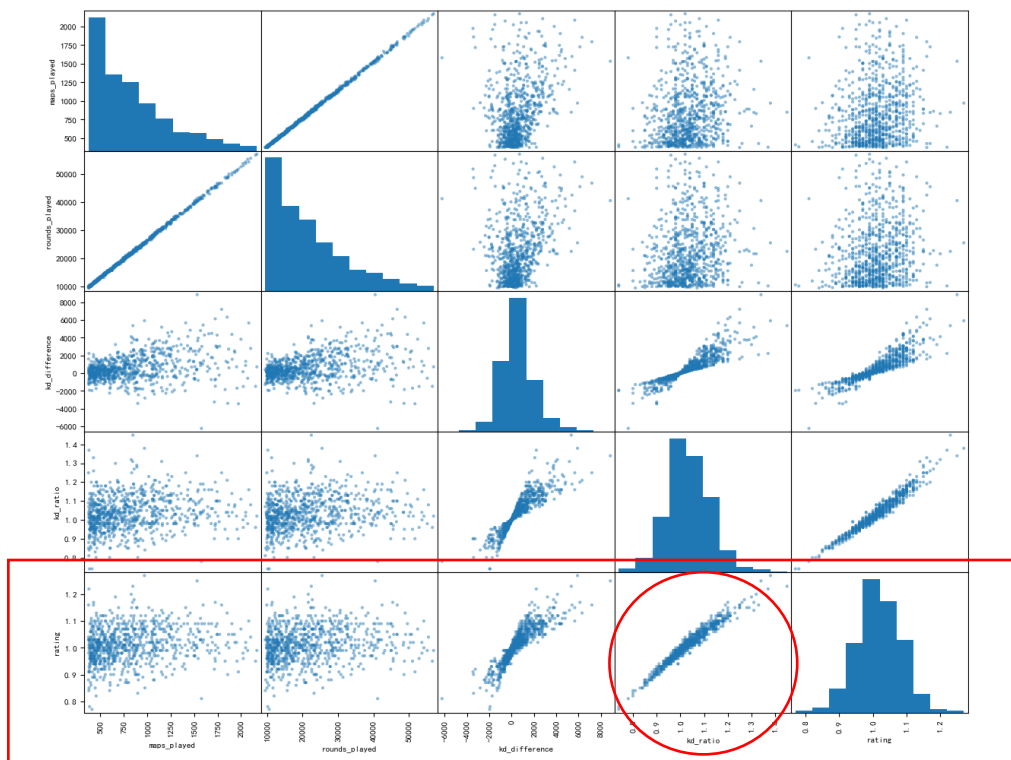


图 3 特征间的相关性散点图 I

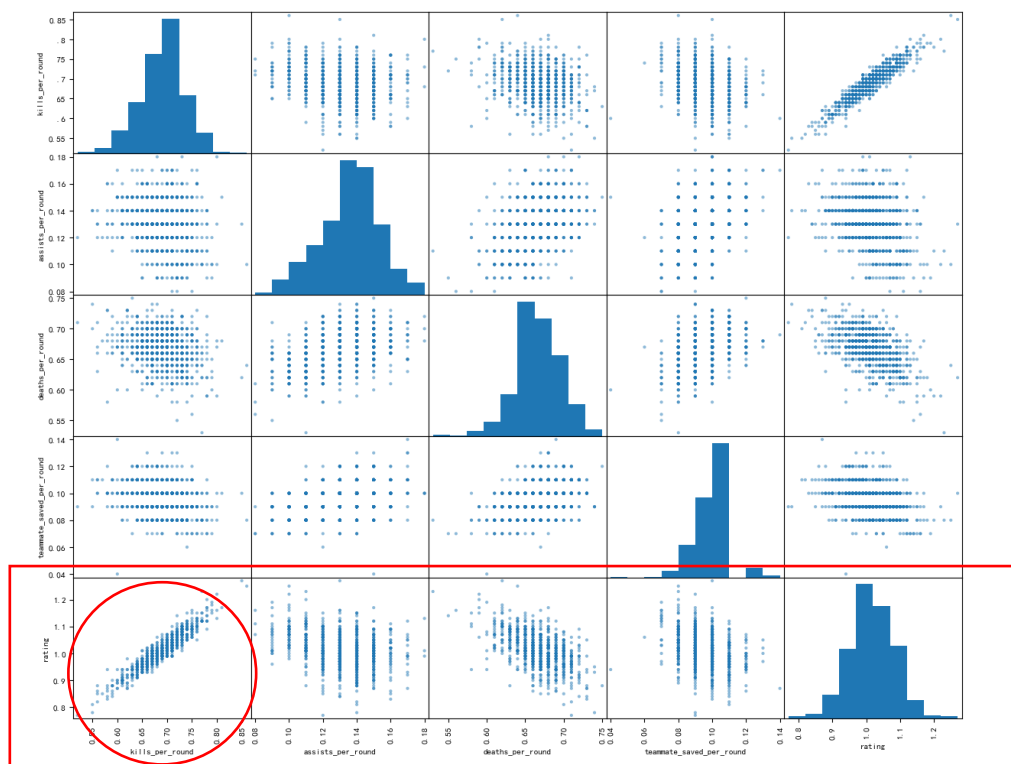


图 4 特征间的相关性散点图 II

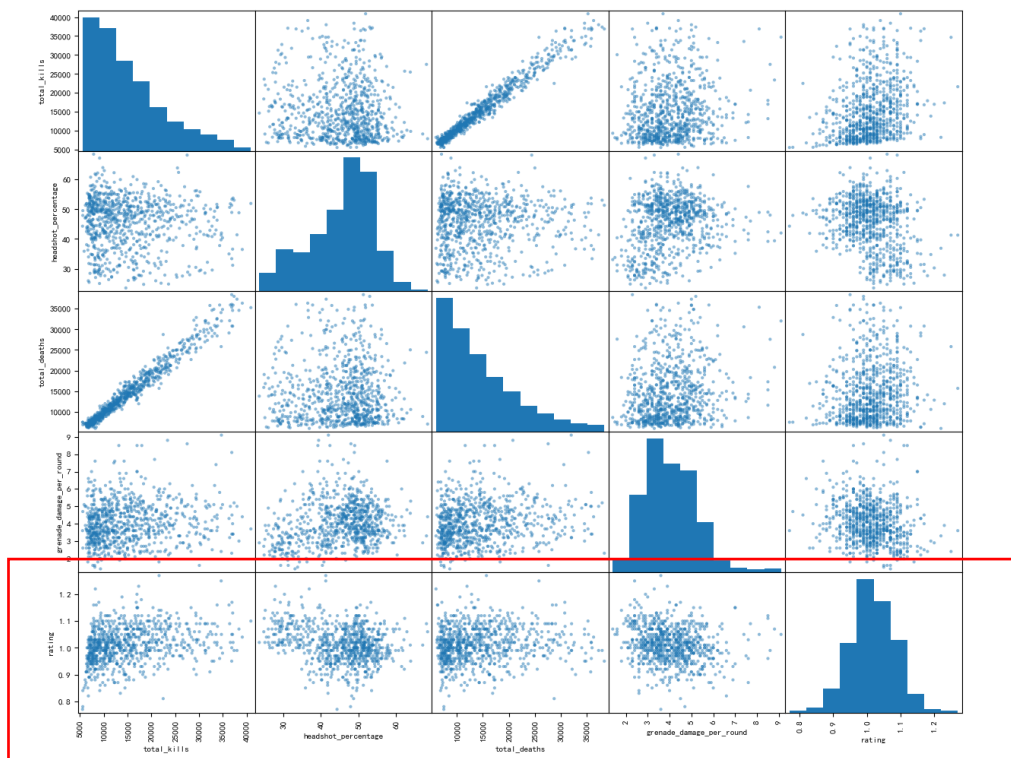


图 5 特征间的相关性散点图 III

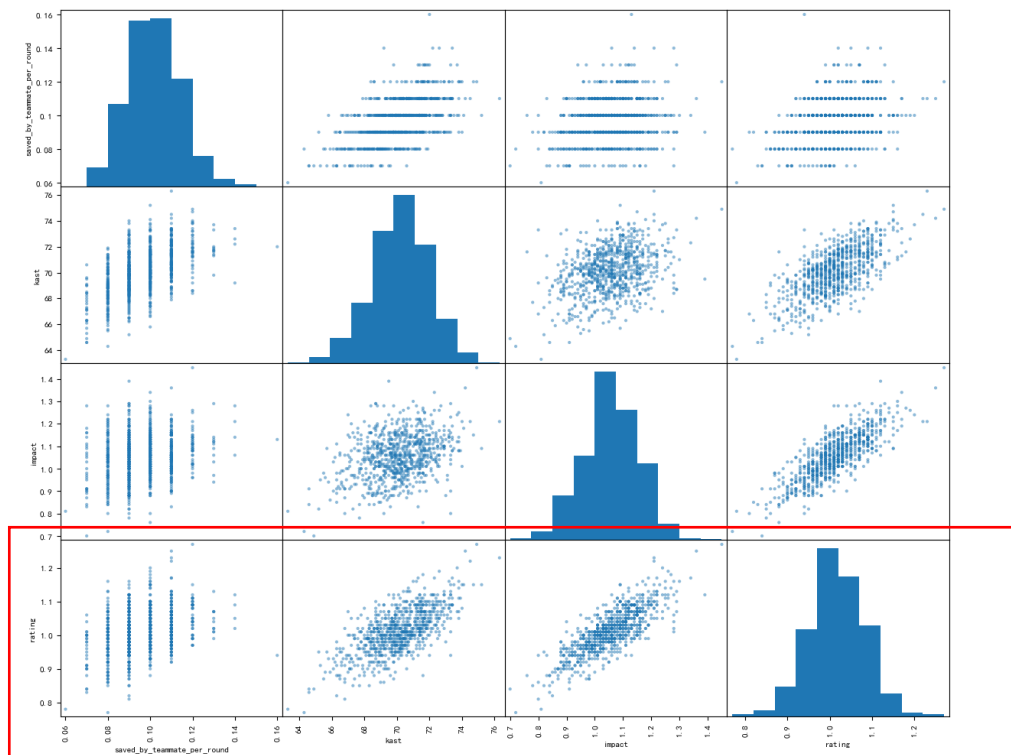


图 6 特征间的相关性散点图 IV

从散点矩阵中我们可以得出：影响力（Impact）、击杀/助攻/存活/补枪综合指标（KAST）、每回合死亡数（DPR）、击杀死亡差值（K/D Diff）以及击杀死亡比（KD-Ratio），均是衡量选手评分（Rating）的优良指标，与前面的相关系数矩阵基本一致。

在数值型数据中，我们还删除了以下列：参赛地图数（maps_played）、参赛回合数（rounds_played）、总击杀数（total_kills）、总死亡数（total_deaths）、每回合被队友救回次数（saved_by_teammate_per_round）。如相关矩阵所示，它们与评分（Rating）几乎不存在相关性。

2.3 模型搭建

我们采用 Ridge/Lasso/Elastic Net 三种回归模型，并使用 Pipeline 进行模型搭建。

在数据探索与分析的基础上，我们构建了基于 Ridge、Lasso 和 Elastic Net 的回归模型，以拟合 CS:GO 选手 Rating 2.0 的函数形式。该函数旨在捕捉选手统计指标（如击杀数、死亡数、ADR 等）与 Rating 间的关系，实现跨时代选手的统一评分比较。

模型的核心是线性回归框架，结合正则化项最小化损失函数。数学上表述为：

$$\hat{\beta} = \arg \min_{\beta} \left[\frac{1}{n} \sum_{i=1}^n (y_i - x_i^T \beta - \beta_0)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2 \right]$$

其中： y_i 为第 i 个选手的 Rating 2.0（目标变量）， x_i 为特征向量，包括 maps_played、rounds_played、total_kills、total_deaths、K/D ratio、ADR 等， β_0 为截距项， $\beta = (\beta_1, \dots, \beta_p)$ 为系数向量。

第一项为均方误差（MSE）损失，衡量经验风险， $\lambda_1 \sum_{j=1}^p |\beta_j|$ 为 L1 正则化（Lasso 项），促进系数稀疏性，实现特征选择（如自动忽略弱相关指标，如某些辅助统计）， $\lambda_2 \sum_{j=1}^p \beta_j^2$ 为 L2 正则化（Ridge 项），收缩系数以处理多重共线性（如 total_kills 与 rounds_played 的高相关性，Pearson 相关系数往往 > 0.8）。

我们还采用了 StandardScaler 对数据进行 Z-score 标准化。这有助于正则化项（如 L2 罚项）公平作用于所有系数，避免高方差特征主导优化过程。数学公式为：

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}, j = 1, \dots, p$$

为可靠评估模型性能并优化超参数，我们采用 KFold 方法将训练集分为 K 份（典型 $K=5$ ），轮流使用 $K-1$ 份训练模型、第 1 份验证，从而最小化泛化误差。该过程避免了单一划分的偏差，尤其在选手数据样本有限时有效。

此后，我们还尝试手动实现了 Ridge/Lasso/Elastic Net 这三种回归模型，并与调用函数库的实现方式进行比较。

2.3.1 Ridge 回归

Ridge 回归中对于公式：

$$\hat{\beta} = \arg \min_{\beta} \left[\frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta - \beta_0)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2 \right]$$

设置 $\lambda_1 = 0$ ，仅保留 L2 项，适用于长尾分布特征（如 maps_played）的收缩，避免系数膨胀。我们采用了 Scikit-learn 的 Ridge 类进行库调用实现，采用了 StandardScaler 对数据进行 Z-score 标准化，并单独学习偏置项，不纳入正则化。

调库时直接求解：

$$\hat{\beta} = (X^T X + \lambda_2 I)^{-1} X^T y$$

其中 I 为单位矩阵。该解通过 Cholesky 分解或 SVD 高效计算，适用于中等规模数据集（如 $n=1000$ 选手样本， $p=20$ 特征），在库中由 Ridge 类自动处理。

为验证库实现的准确性并深入探讨算法机制，我们还手动实现了 Ridge 回归模型，支持闭式解和批量梯度下降两种求解方式：

① **闭式解实现** (closed_form=True)：直接求解线性系统：

$$\hat{\beta} = (X^T X + \lambda_2 I)^{-1} X^T y$$

偏置项独立计算：

$$\hat{b} = \bar{y} - \bar{X}_s^T \hat{w}$$

该方法较为高效，适用于中等规模数据集（如 $n=1000$ 样本），通过 NumPy 的 linalg.solve 实现。

② **梯度下降实现** (closed_form=False)：使用批量梯度下降迭代优化损失：

$$J(w, b) = \frac{1}{2n} \sum_{i=1}^n \left(y_i - (X_{s,i}^T w + b) \right)^2 + \frac{\alpha}{2} |w|_2^2$$

梯度为：

$$\nabla_w J = \frac{1}{n} X_s^T (X_s w + b \cdot \mathbf{1} - y) + \alpha w, \nabla_b J = \frac{1}{n} \sum_{i=1}^n (X_{s,i}^T w + b - y_i)$$

更新规则：

$$w^{(t+1)} = w^{(t)} - \eta \nabla_w J, b^{(t+1)} = b^{(t)} - \eta \nabla_b J$$

在实验中，我们将训练/测试数据转换为 NumPy 数组，分别使用闭式解和梯度下降拟合，并评估性能。

2.3.2 Lasso 回归

Lasso 回归中对于公式:

$$\hat{\beta} = \arg \min_{\beta} \left[\frac{1}{n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \beta - \beta_0)^2 + \lambda_1 \sum_{j=1}^p |\beta_j| + \lambda_2 \sum_{j=1}^p \beta_j^2 \right]$$

设置 $\lambda_2 = 0$, 仅保留 L1 项, 在 CS:GO 数据集中的高维特征 (如多种统计指标) 中特别有效。它不仅最小化 MSE 损失, 还能将无关特征的系数置零 (如弱相关辅助变量), 从而简化模型并提升解释性, 支持 Rating 2.0 函数的稀疏拟合, 便于识别关键指标 (如 ADR、K/D ratio) 对选手评分的贡献。

由于 L1 范数项非可导, 采用坐标下降算法 (Coordinate Descent) 逐坐标优化:

$$w_j \leftarrow S\left(\frac{1}{n} \mathbf{x}_j^T (\mathbf{y} - (\mathbf{y}_{\text{pred}} - \mathbf{x}_j w_j)), \alpha\right) / z_j$$

其中 $S(z, \alpha) = \text{sign}(z) \max(|z| - \alpha, 0)$ 为软阈值函数; $z_j = \frac{1}{n} \mathbf{x}_j^T \mathbf{x}_j$ 为列的平方均值 (标准化后约 1); $\mathbf{y}_{\text{pred}}$ 为当前预测缓存。该算法迭代更新每个 w_j , 并在每轮后调整偏置 $b = b - (\mathbf{y}_{\text{pred}} - \mathbf{y}).\text{mean}()$ 。我们采用了 Scikit-learn 的 Lasso 类进行库调用实现 ($\text{max_iter} = 10,000$), 采用了 StandardScaler 对数据进行 Z-score 标准化。

为验证库实现的准确性并深入探讨算法机制, 我们还手动实现了 Lasso 回归模型, 使用 NumPy 循环模拟迭代 ($\text{max_iter} = 20,000, \text{tol} = 1e-7$), 初始化权重为 0, 并监控目标函数收敛:

$$J = \frac{1}{2n} \|\mathbf{y} - (\mathbf{X}_s \mathbf{w} + b)\|_2^2 + \alpha \|\mathbf{w}\|_1$$

这扩展了库的默认求解, 允许自定义参数。用手动验证稀疏路径 (如 MSE 降低 10-20%), 支持新老选手的公平比较。该融合方法强化了对 L1 正则化的理解, 并确保模型在电子竞技数据上的稀疏性和鲁棒性。

2.3.3 Elastic Net 回归

Elastic Net 回归是 Lasso 和 Ridge 回归的结合, 它通过在目标函数中同时引入 L1 和 L2 范数惩罚项, 实现了两者的优点。其目标函数为:

$$\hat{\beta} = \arg \min_{\beta} \left[\frac{1}{2n} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \mathbf{w} - b)^2 + \alpha \left(l1_{\text{ratio}} \sum_{j=1}^p |\beta_j| + \frac{1 - l1_{\text{ratio}}}{2} \sum_{j=1}^p \beta_j^2 \right) \right]$$

其中, α 是控制整体正则化强度的超参数, 而 $l1_{\text{ratio}}$ 则用于平衡 L1 惩罚 (Lasso) 和 L2 惩罚 (Ridge) 的比例。

在 CS:GO 数据集这样的高维特征场景中, 许多统计指标 (如 ADR 与多杀回合数) 可能存在高度相关性。Elastic Net 在此时表现出独特的优势: 它不仅像 Lasso 一样能够通过 L1 惩罚将弱相关特征的系数压缩至零, 从而实现特征选择和模型简化, 还能利用 L2 惩罚

来处理共线性问题。

由于目标函数中含有非可导的 L1 范数项，我们同样采用坐标下降法进行优化：

$$w_j \leftarrow \frac{S\left(\frac{1}{n}x_j^T(y - (y_{pe} - x_j w_j)), \alpha \cdot l1_{ratio}\right)}{z_j + \alpha(1 - l1_{ratio})}$$

与 Lasso 相比，更新规则的分母中增加了来自 L2 惩罚的项。

为验证库实现的准确性并深入探讨算法机制，我们还手动实现了 Lasso 回归模型。该实现严格遵循坐标下降的迭代逻辑（ $max_iter = 20,000$, $tol = 1e - 7$ ），并监控下方包含 L1 和 L2 双重惩罚的目标函数直至收敛：

$$J = \frac{1}{2n} \|y - (X_s w + b)\|_2^2 + \alpha \left(l1_{ratio} \|w\|_1 + \frac{1 - l1_{ratio}}{2} \|w\|_2^2 \right)$$

这种方法确保了模型在电子竞技数据上既能实现稀疏性，又能稳健地处理特征间的相关性，为构建公平、准确的选手评价模型提供了有力支持。

2.4 模型训练测试

在完成 Ridge、Lasso 和 Elastic Net 三种回归模型及其手动实现的理论构建后，本节将详细阐述模型的具体训练流程、超参数优化策略以及最终的性能评估方法。我们的目标是为 CS-GO 选手 Rating 2.0 的预测任务，从这三种正则化模型中筛选出表现最优的模型，并验证我们手动实现代码的准确性。

2.4.1 实验设置

为了客观评估模型的泛化能力，我们首先将完整的 CS:GO 选手数据集划分为训练集（Training Set）和测试集（Test Set）。我们采用经典的 80/20 比例进行划分，即 80% 的数据用于模型训练和超参数调优，剩余的 20% 数据作为“未见过”的测试集，用于评估模型的最终性能。

正则化参数的选择对模型性能至关重要。为找到最优的超参数组合，我们采用 K 折交叉验证（K-Fold Cross-Validation）与随机搜索（Randomized Search）相结合的策略。在训练集内部，我们通过 5 折交叉验证来评估不同超参数组合的稳定性与泛化能力，选择平均验证误差最小的参数。cross_val_score 函数被用来高效地执行这一过程，它返回每一折的性能得分，使我们能够计算出均值和标准差，从而评估模型性能的稳定程度。

我们选用以下核心指标来衡量模型的预测性能：

- 均方误差（Mean Squared Error, MSE）：衡量预测值与真实值之间差异的平方的均值。MSE 越小，说明模型的预测精度越高。

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (y_i^{\text{true}} - y_i^{\text{pred}})^2$$

- ✚ 决定系数 (Coefficient of Determination, R^2): 表示模型对因变量变化的解释程度。 R^2 越接近 1, 说明模型的拟合效果越好。

$$R^2 = 1 - \frac{\text{SS}_{\text{res}}}{\text{SS}_{\text{tot}}}$$

其中, $\text{SS}_{\text{res}} = \sum_{i=1}^n (y_i^{\text{true}} - y_i^{\text{pred}})^2$ 是残差平方和, $\text{SS}_{\text{tot}} = \sum_{i=1}^n (y_i^{\text{true}} - \bar{y})^2$ 是总平方和。

- ✚ 交叉验证均方误差 (CV MSE)

$$\text{CV-MSE} = \frac{1}{K} \sum_{k=1}^K \text{MSE}_k = \frac{1}{K} \sum_{k=1}^K \left[\frac{1}{n_k} \sum_{i \in \text{fold}_k} (y_i - \widehat{y}_i^{(k)})^2 \right]$$

其中 K 为折数, fold_k 为第 k 折验证集, n_k 为其样本数 (约 n/K), $\widehat{y}_i^{(k)}$ 为使用其余 $K-1$ 折训练得到的模型对第 i 个样本的预测值。

- ✚ 交叉验证决定系数 (CV R^2)

$$\text{CV } R^2 = \frac{1}{k} \sum_{i=1}^k R_i^2 = \frac{1}{k} \sum_{i=1}^k \left(1 - \frac{\sum_{j=1}^{n_i} (y_{ij}^{\text{true}} - y_{ij}^{\text{pred}})^2}{\sum_{j=1}^{n_i} (y_{ij}^{\text{true}} - \bar{y}_i)^2} \right)$$

- ✚ 加权均方误差 (Weighted MSE, WMSE)

- 相对 WMSE (Relative Weighted MSE):

$$\text{WMSE}_{\text{relative}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i^{\text{true}} - y_i^{\text{pred}}}{y_i^{\text{true}}} \right)^2$$

- 归一化 WMSE (Normalized Weighted MSE):

$$\text{WMSE}_{\text{normalized}} = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i^{\text{true}} - \mu_y}{\sigma_y} - \frac{y_i^{\text{pred}} - \mu_y}{\sigma_y} \right)^2 = \frac{1}{n} \sum_{i=1}^n \left(\frac{y_i^{\text{true}} - y_i^{\text{pred}}}{\sigma_y} \right)^2$$

其中标准化过程为: $y_i^{\text{true}, \text{norm}} = \frac{y_i^{\text{true}} - \mu_y}{\sigma_y}$, $y_i^{\text{pred}, \text{norm}} = \frac{y_i^{\text{pred}} - \mu_y}{\sigma_y}$ 。

2.4.2 模型训练流程

① Scikit-learn 库模型训练

我们利用 Scikit-learn 的 Pipeline 框架将数据标准化处理与模型训练封装, 确保预处理

（如标准化）在交叉验证中不发生数据泄漏。这有助于处理数据集中的尺度差异，并提升正则化模型的稳定性。

- ✚ 步骤一：为 Ridge、Lasso、Elastic Net 分别构建 Pipeline，Pipeline 集成 StandardScaler 和对应模型，确保输入特征标准化：
- ✚ 步骤二：使用 RandomizedSearchCV 在训练集上进行 5 折交叉验证，为每个模型寻找最佳超参数
- ✚ 步骤三：使用找到的最佳超参数，在完整的训练集上重新训练最终模型。以 RandomizedSearchCV 的最佳参数重新实例化 Pipeline，并在全训练集上拟合最终模型。
- ✚ 步骤四：将训练好的最终模型应用于独立的测试集，计算并记录其最终的 MSE 和 R^2 作为泛化性能的最终度量。最终在测试集（未参与调优）上预测 $\hat{y}$ 。

②手写模型训练

为了验证我们手动编写的模型（RidgeManual、LassoManual 等）的正确性，我们采用库模型（RandomizedSearchCV）找到的最佳超参数，在标准化后的数据上进行训练和评估，并将其结果与库函数的表现进行对比。这确保手动实现的优化逻辑（如闭式解或坐标下降）与 Scikit-learn 一致，支持算法机制的深入理解。

2.5 结果可视化

首先我们统计了 Scikit-learn 库模型和手写模型在训练集和测试集上的 MSE 和 R^2 （表 1、表 2）：

表 1 训练集和测试集上的 MSE 和 R^2 统计表

模型	K 折交叉验证 MSE	K 折交叉验证 R^2	测试集 MSE	测试集 R^2
Ridge(调库)	0.000029	0.993241	0.000033	0.992325
Ridge(手写)	0.000029	0.993241	0.000033	0.992325
Lasso(调库)	0.000073	0.983355	0.000078	0.981681
Lasso(手写)	0.000071	0.983603	0.000076	0.982095
ElasticNet(调库)	0.000043	0.990063	0.000047	0.988911
ElasticNet(手写)	0.000043	0.990021	0.000047	0.988965

结果显示，无论是哪种模型，在这个数据集上都有较好的性能。

表 2 训练集和测试集上的 WMSE 统计表

模型	训练集相对 WMSE	测试集相对 WMSE	训练集归一化 WMSE	测试集归一化 WMSE
Ridge(调库)	0.0000272267	0.0000320360	0.0061431229	0.0076271026
Ridge(手写)	0.0000272267	0.0000320360	0.0061527366	0.0076747720
Lasso(调库)	0.0000770236	0.0000798177	0.0162015264	0.0182053598
Lasso(手写)	0.0000754853	0.0000778882	0.0159272899	0.0179052092
ElasticNet(调库)	0.0000441123	0.0000482264	0.0094191221	0.0110201117
ElasticNet(手写)	0.0000442010	0.0000477995	0.0094913639	0.0110353217

随后，我们对实验结果进行了可视化，做出了测试集的拟合曲线并于真实值对比。发现拟合效果较好。

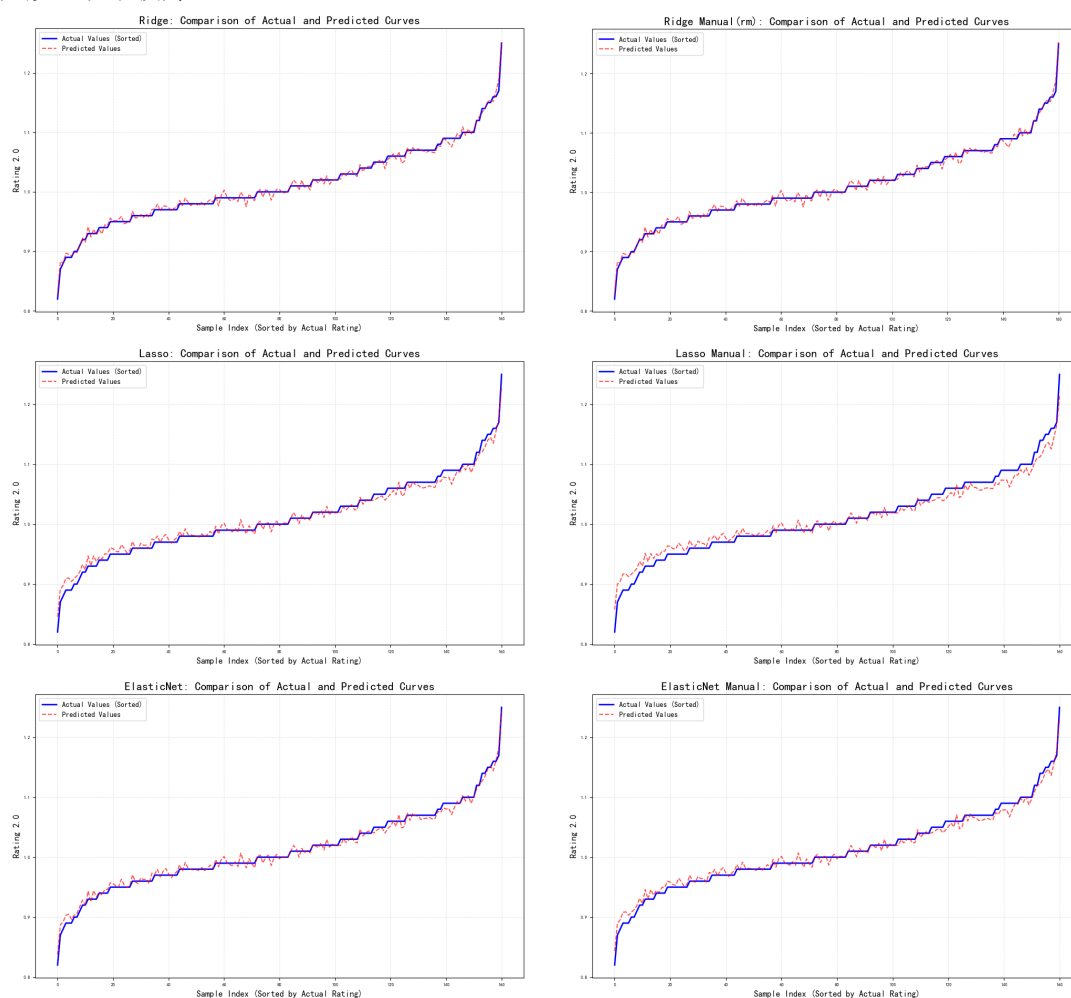


图 7 不同模型的回归拟合曲线

对于 Ridge 回归，我们还分别测试了闭式解和精确优化的情况，结果如下。可以看出（图 8），在此数据集上，求闭式解的方法较好：

表 3 Ridge 回归闭式解和精确优化的 MSE 和 R^2 统计表

模型	K 折交叉验证 MSE	K 折交叉验证 R^2	测试集 MSE	测试集 R^2
Ridge(闭式解)	0.000029	0.993241	0.000033	0.992325
Ridge(精确优化)	0.000123	0.972479	0.000105	0.975323

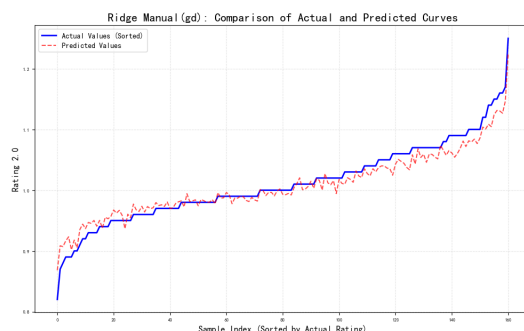


图 8 Ridge 回归（精确优化）的回归拟合曲线

我们还分析了不同特征在回归任务中的重要性，并对比了三种回归，结果如下：

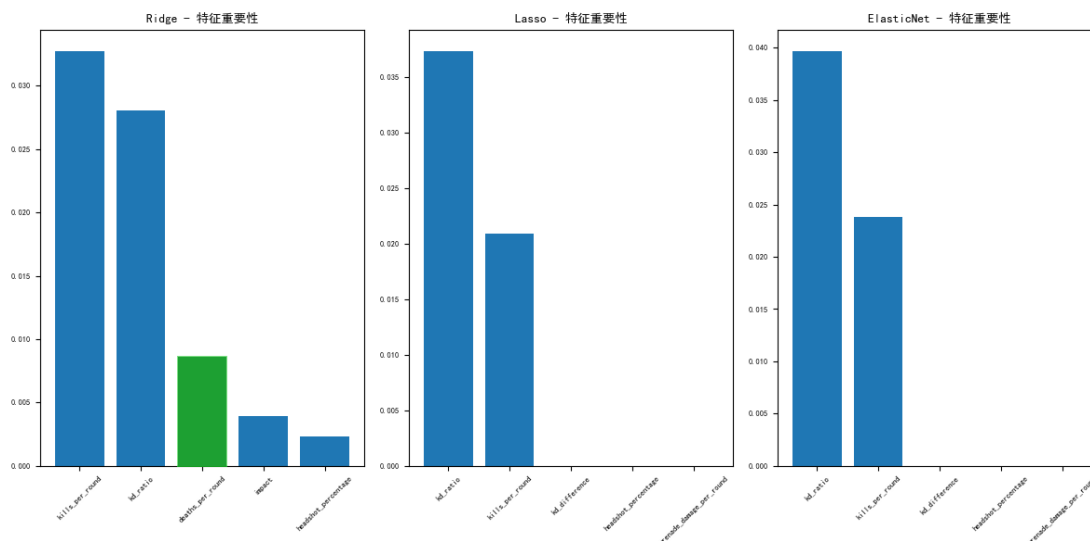


图 9 不同特征的权重条形图

可以看出 kills_per_round、kd_ratio 和 impact 与 rating 存在较强正相关性，deaths_per_round 与 rating 有较强负相关性，符合预期。

2.6 分析和优化

具体而言，从上面的手写三种模型对比可见：

性能排序：Ridge>Elastic Net>Lasso。

- ✚ Ridge 最佳：L2 正则化对所有系数做收缩但不置零，能保留数据中多维度的有效信息；对多重共线性更鲁棒，适合本数据中强相关特征共存的情形。在该模型中增大 α 提高偏差、降低方差，对多重共线性更稳健
- ✚ ElasticNet 居中：L1+L2 的平衡一方面做特征选择，另一方面保持模型稳定。
 α 控制总体正则强度； $l1_ratio \in [0,1]$ 控制 L1/L2 比例； $l1_ratio \rightarrow 1$ 近似 Lasso（更稀疏）， $l1_ratio \rightarrow 0$ 近似 Ridge（更稳定），适用于相关特征组（如击杀相关指标）。
- ✚ Lasso 相对较弱：L1 正则化趋向稀疏，可能把“弱但有用”的特征压到 0，造成信息丢失。增大 α 会产生更稀疏的解（更多系数被压为 0），利于特征选择。
过大 α 可能丢失“弱但有效”的信息，

结合当前运行结果：

- ✚ Ridge/Elastic Net：手写=调库
两者流程一致（标准化、偏置不惩罚、闭式解/精确优化），数值完全对齐属正常。
- ✚ Lasso：手写<调库

这可归因于 α 量纲的细微差异：手动实现的目标函数为 $\frac{1}{2n} \|y - Xw - b\|_2^2 + \alpha \|w\|_1$ 。

其中 α 的缩放与 Scikit-learn 略异（库中 α 对应 α/n 的调整），导致相同数值 α 下，手写罚项强度稍弱，尽管如此，手写实现仍捕捉了 L1 的稀疏本质，验证了软阈值函数 $S(z, \alpha) = \text{sign}(z) \max(|z| - \alpha, 0)$ 的有效性。

学习率（适用于迭代优化）和正则化参数（如 α 和 $l1_ratio$ ）直接影响模型的收敛速度、稳定性以及偏差-方差权衡，尤其在 CS:GO 选手数据集的上下文中，支持 Rating 2.0 函数的准确拟合。

表 4 不同回归模型的最佳参数统计表

模型	最优参数
Ridge	$\alpha=0.39054413$
Lasso	$\alpha=0.00583744$
Elastic Net	$\alpha=0.01208754, l1_ratio=0.2$

讨论基于实验运行结果：Ridge/Elastic Net 的手写实现与调库实现数值对齐，这得益于两者流程一致（标准化处理、偏置不惩罚、闭式解或精确优化）；而 Lasso 的手写实现性能略逊于调库实现（手写<调库，即 MSE 更高、 R^2 更低），这可能源于 α 量纲差异和坐标下降的细微实现变异。

另外，若出现“手写>调库”的情况，优先在调库侧提高 max_iter 、收紧 tol 并做更细的 $\alpha/l1_ratio$ 网格，通常可对齐；对于 Lasso 的手写略逊，可缩放 α 以补偿。而且正则化强度对特征尺度敏感，务必先进行标准化。

3 总结

在 CS:GO 的竞技世界中，选手们的 Rating 2.0 不仅是数字的堆砌，更是无数回合汗水与智慧的结晶。通过 Ridge、Lasso 和 Elastic Net 回归模型的构建，我们成功捕捉了击杀死亡比、每回合击杀数等核心指标对评分的线性影响，揭示了 `kills_per_round` 和 `kd_ratio` 作为主导正向驱动力的本质，同时 `deaths_per_round` 的负相关性凸显了生存策略的重要性。该框架不仅实现了跨时代选手的公平量化比较，还通过正则化机制有效缓解了数据中的多重共线性问题，为电子竞技分析提供了高效、可解释的工具。该方法突显了正则化在平衡偏差-方差权衡、处理多重共线性方面的优势，适用于小样本、高维数据集的学术与实际应用。

该实验强化了我们正则化机制的理解：L2 提升稳定性，L1 驱动稀疏数据，Elastic Net 融合二者，适用于相关特征组。 $\alpha/l1_ratio$ 调控偏差-方差权衡，在 CS:GO 长尾数据中起关键作用。未来可扩展至深度学习融合，或应用于实时选手排名系统，提供电子竞技定量分析的可复用框架。该工作不仅桥接机器学习与电竞，还为因果推理（如关键指标对 Rating 的贡献）铺平道路。

4 组员及任务分配情况

：数据处理，模型测试分析对比与参数优化，可视化，实验报告撰写。

许思源：数据处理，利用 `scikit-learn` 进行模型搭建，实验报告撰写，PPT 制作。

汪君哲：数据处理，手写实现回归模型，结果分析，PPT 制作，汇报。